

基于Tsransformer-GCN的源代码漏洞检测方法

梁 辰, 王奕森*, 魏 强, 杜 江

(信息工程大学 网络空间安全学院, 郑州 450001)

(* 通信作者电子邮箱 851067568@qq.com)

摘 要: 针对现有的基于深度学习的源代码漏洞检测方法存在目标代码语法和语义缺失严重以及神经网络模型对目标代码图点(边)权重分配不合理等问题, 提出一种基于代码属性图(CPG)与自适应图卷积网络(AT-GCN)的源代码漏洞检测方法 VulATGCN。该方法使用 CPG 对源代码进行表征, 结合 CodeBERT 进行节点向量化, 并通过图中心性分析提取深层次结构特征, 从而多维度地捕捉代码的语法和语义信息。之后, 结合 Transformer 自注意力机制善于捕捉长距离依赖关系和图卷积网络(GCN)善于捕捉局部特征的优势设计 AT-GCN 模型, 从而实现对不同重要性区域特征的融合学习和精确提取。在真实漏洞数据集 Big-Vul 和 SARD 上的实验结果表明, 所提方法 VulATGCN 的平均 F1 分数达到了 82.9%, 相较于 VulSniper, VulMPFF 和 MGVD 等基于深度学习的漏洞检测方法提高了 10.4%~132.9%, 平均提高约 52.9%。

关键词: 源代码漏洞检测; 代码属性图; 图神经网络; 中心性分析; 自注意力机制

中图分类号: TP311 **文献标志码:** A

Source code vulnerability detection method based on Transformer-GCN

LIANG Chen, WANG Yisen*, WEI Qiang, DU Jiang

(School of Cyberspace Security, Information Engineering University, Zhengzhou Henan 450001, China)

Abstract: The existing deep learning-based methods for source code vulnerability detection often suffer from severe loss of syntax and semantics in target code, and neural network models allocating weights to the graph nodes (edges) in target code unreasonably. To address these issues, a method named VulATGCN for detecting source code vulnerabilities was proposed on the basis of Code Property Graph (CPG) and Adaptive Transformer-Graph Convolutional Network (AT-GCN). In the method, CPG was used to represent source code, CodeBERT was combined for node vectorization, and graph centrality analysis was employed to extract deep structural features, thereby capturing the code's syntax and semantic information in multi-dimensional way. After that, AT-GCN model was designed by integrating strengths of Transformer-based self-attention mechanism, which excels at capturing long-range dependencies, and Graph Convolutional Network (GCN), which is proficient at capturing local features, thereby realizing fusion learning and precise extraction of features from regions with different importance. Experimental results on real vulnerability datasets Big-Vul and SARD show that the proposed method VulATGCN achieves an average F1 score of 82.9%, which is 10.4% to 132.9% higher than deep learning-based vulnerability detection methods such as VulSniper, VulMPFF, and MGVD, with an average increase of approximately 52.9%.

Key words: code vulnerability detection; Code Property Graph (CPG); Graph Neural Network (GNN); centrality analysis; self-attention mechanism

0 引言

软件漏洞是指软件系统中存在的可能被攻击者利用的缺陷或弱点。随着软件系统日益复杂化, 漏洞检测变得越来越具有挑战性^[1]。传统的静态分析和动态测试方法在处理大规模复杂软件时通常效率低下且误报率高。近年来, 基于深度学习的漏洞检测方法展现出巨大的潜力, 但如何有效地表示和学习代码的语法和语义特征仍是一个关键问题^[2]。

基于深度学习的漏洞检测研究在近几年提出了各种解决方案。例如, 文献[3]提出了结合代码语法和语义的代码综合表示学习方法, 文献[4]通过图门控神经网络(Graph Gated Neural Network, GGNN)对图表征的源代码进行特征嵌入。然而, 这些方法在捕捉复杂代码结构特征和权衡代码整

体特征与局部特征等方面, 仍然存在一定的局限性。

为解决这些问题, 本文提出了一种基于代码属性图(Code Property Graph, CPG)^[5]和自适应图卷积网络(Adaptive Transformer-Graph Convolutional Network, AT-GCN)的漏洞检测方法 VulATGCN。VulATGCN 首先将源代码转换为 CPG, 以统一的图结构表示代码的语法和语义特征。然后, 使用 CodeBERT (Code Bidirectional Encoder Representations from Transformers)^[6]对 CPG 中的节点进行向量化表征, 以捕捉更深层次的代码语义信息。在此基础上, 通过图中心性分析提取高层次的图结构特征, 将图中心性分析特征与节点特征结合, 增强对代码整体结构的理解。最后, 设计 AT-GCN 模型, 融合 Transformer 自注意力机制和图卷积网络(Graph Convolutional Network, GCN)的优势, 实现

收稿日期: 2024-07-17; 修回日期: 2024-10-31; 录用日期: 2024-10-31。 基金项目: 河南省重点研发专项(221111210300)。

作者简介: 梁辰(2000—), 男, 安徽合肥人, 硕士研究生, 主要研究方向: 软件成分分析; 王奕森(1990—), 男, 河南沈丘人, 副教授, 博士, 主要研究方向: 网络安全; 魏强(1979—), 男, 江西南昌人, 教授, 博士, 主要研究方向: 软件安全、工业控制系统安全; 杜江(1990—), 男, 河南郑州人, 博士研究生, 主要研究方向: 二进制代码相似性。

对代码特征的全面学习。

在Big-Vul^[7]和软件保障参考数据集(Software Assurance Reference Dataset, SARD)^[8]两个漏洞数据集上的实验结果表明,本文方法VulATGCN的平均F1分数达到82.9%,相较于LineVD^[9]、MGVD^[10]和文献[4]方法等基于深度学习的漏洞检测方法,平均提高了约52.9%。

本文的主要工作如下:

1)提出了一种基于CPG并结合CodeBERT进行节点向量化的代码表示方法VulATGCN,可全面捕捉代码的语法和语义信息。

2)引入图中心性分析,提取高层次的图结构特征,增强模型对代码整体结构的理解。

3)实现了AT-GCN模型,融合Transformer和GCN的优势,实现对不同重要性区域特征的融合学习和精确提取。

1 相关工作

近年来,由于深度学习在建模能力和特征学习方面的显著优势,它开始被广泛应用于源代码漏洞检测领域,帮助专家构建漏洞特征和进行漏洞检测。

本章先介绍程序静态分析领域相关基础知识,再依据基于深度学习的源代码漏洞检测方法预处理阶段中对源代码的表征方法,介绍近年来基于深度学习的源代码漏洞检测方法的相关工作,包括基于token序列和基于图的表征方法。

1.1 程序静态分析

本节介绍程序静态分析领域内常用的源代码的表征形式,包括:

1)抽象语法树(Abstract Syntax Tree, AST):AST用树状结构表现源代码的语法,可反映代码的语法结构和元素之间的关系。AST生成过程中,首先经过词法分析将代码分解为一系列的token,然后通过语法分析,根据编程语言的语法规则将这些标记组合成树。AST偏向描述代码的语法结构,是构建其他代码表征图的基础。

2)控制流图(Control Flow Graph, CFG):是代码的一种有向图表示,主要反映代码执行过程中的控制流程。CFG的节点表示函数中的基本块(一段没有分支的连续代码),边表示控制流的方向。CFG是编译器优化和静态分析的重要工具,可帮助理解代码的执行流程和代码块间控制关系。

3)数据依赖:数据依赖描述代码中对数据的定义、赋值和数据使用关系,主要反映数据在代码中的流动和使用情况。数据依赖作为静态分析和优化中的关键概念,有助于识别代码中的数据流和潜在的关键节点。

4)控制依赖:描述代码中语句的条件关系,主要反映代码中的控制逻辑和执行顺序,包括控制语句(如if、while等)和受这些控制语句影响的语句之间的依赖关系。与数据依赖类似,控制依赖常搭配数据依赖共同对代码进行分析,帮助理解和优化代码,识别关键节点。

5)程序依赖图(Program Dependence Graph, PDG):是程序的一种图形表示。在PDG中,每个节点代表源代码中的语句或表达式,而边则表示这些语句或表达式之间的依赖关系,包括数据依赖、控制依赖以及输出依赖等。PDG侧重于表示程序中的各种依赖关系,帮助静态分析工具进行更精确的代码分析和理解。

6)代码属性图:CPG旨在统一结构以捕获尽可能多的程序信息和语义,是一个多层次、多维度的图表示法,它合并AST、CFG、PDG等图,并在此基础上添加了丰富的属性信息。CPG用节点表示函数、语句、变量等,且每个节点都有一个唯

一标识符,同时节点的边关系包括数据边、控制边和调用边。数据边表示代码中的变量依赖关系,控制边表示代码中的控制依赖关系,调用边表示代码中的函数调用关系。CPG是目前语义信息最全面的抽象图结构。

1.2 基于token序列表征的漏洞检测方法

在源代码的解析过程中,编译器将代码转换为一连串有意义的token,包括关键词、运算符等。这些token序列能有效地表示代码的词法结构和语义,同时便于向量化处理。因此,通过各种方法提取源代码中的词法关系和语义信息,将token组合成token序列,是对源代码的一种合理抽象表示。

文献[11]提出了基于深度学习的漏洞检测系统VulDeePecker,重点在于通过“code gadget”抽取反映漏洞数据流语义和结构流关系的代码片段。选用CWE(Common Weakness Enumeration)漏洞库中的CWE339和CWE119漏洞代码,通过词法分析和word2vec将code gadget转换向量表征,训练双向长短期记忆(Bidirectional Long Short-Term Memory, BLSTM)网络。

文献[12]提出了 μ VulDeePecker,对VulDeePecker进行了改进以检测漏洞类型。首先使用Joern工具生成依赖图,结合函数调用关系构建系统依赖图,识别关键节点生成code gadget,并根据预定义语法规则创建code attention。这些向量化后的特征输入3个新的BLSTM中,分别提取全局特征和局部特征并进行融合,用于精确漏洞分类器的判断。

1.3 基于图表征的漏洞检测方法

与基于token序列表征的方法相比,基于图的表征可以表征更复杂的代码信息。目前,学术界提出的方法包括使用基于AST的表征、基于CPG的表征、基于PDG的表征等。无论是树结构,还是更复杂的PDG和CPG,在以token表达节点的同时,节点间还存在明显的连接关系和层次划分,可以更好地刻画代码上下文复杂的语法联系和语义流动调用过程。同时由于不同图对代码不同内容的偏重刻画,基于不同图表征的漏洞检测方法有着不完全类似的处理方式和训练检测过程。

文献[13]提出了一种基于长短期记忆(Long Short-Term Memory, LSTM)网络的代码检测方法,通过词法分析构建AST并转换为代码标记序列,生成固定长度的向量特征,并通过池化操作聚合成特征向量。引入计算机视觉中“码本”概念,将token类比为凸点并使用聚类生成码本,以提取全局特征。在18个Android和Firefox应用程序中的预测结果表明,无论是项目内还是跨项目检测,该方法优于传统的软件度量和词袋模型方法。

文献[14]提出了一种基于深度学习的跨域漏洞检测方法VulGDA,该方法通过词法分析、语法分析和AST构建生成CPG,并利用预训练的词嵌入模型进行节点嵌入。使用GGNN从CPG中传播和聚合邻居信息,实现了漏洞特征的稳定转移和识别。采用迁移学习和深度领域适应的特征生成器,在共享特征空间中提升了跨域检测性能。

文献[9]介绍了一个基于深度学习的漏洞检测方法LineVD,该方法使用CodeBERT进行代码分词和函数级语句特征提取,并结合图注意力网络(Graph Attention Network, GAN)学习PDG。通过信息传播机制学习代码的数据和控制依赖,通过共享的神经网络层训练分类器,实现了函数级和代码级代码的联合学习。

DeepWukong^[15]基于代码PDG,使用Doc2Vec将PDG中的每个语句token转换为向量表示,最后将结构化和非结构化信息作为神经网络的输入,实现了深度学习在代码分析中

的应用。

基于 token 的代码表征虽然贴近源代码的语义,但无法保留重要的结构信息如控制流和数据流,限制了漏洞检测的效果。目前的预处理方法已逐步强化结构信息在 token 序列中的体现,通过各种工具和方法提升代码表征质量。相比之下,基于图表征的方法能更全面地捕捉源代码的复杂结构和语义关系,适合深度学习模型学习程序中的漏洞特征。因此,本文选择基于图表征的方式对源代码进行表征,以确保

对真实漏洞的精准检测。

2 VulATGCN 框架设计

本文方法 VulATGCN 首先基于中心性方法强化 CPG 对函数的结构化表示,对 CPG 特征进行增强;同时设计并实现一个基于 CPG 的自适应图卷积网络,旨在增强对 CPG 特征分析能力的同时提升软件漏洞检测的准确性和效率。VulATGCN 的总体架构包括 6 个主要模块,形成一个端到端的漏洞检测流程,如图 1 所示。

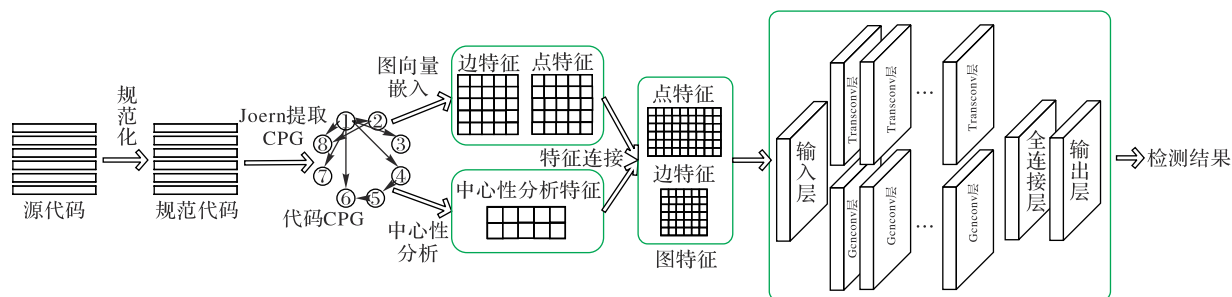


图 1 VulATGCN 的总体流程

Fig. 1 Overall flow of VulATGCN

代码规范化模块:对源代码进行预处理,去除无关信息,统一变量和函数名。

CPG 提取模块:利用 Joern 工具从规范化代码中提取代码属性图。

图向量嵌入模块:使用 CodeBERT 模型将 CPG 的节点和边转换为向量表示。

图特征提取模块:计算图的中心性指标,增强节点特征。

图神经网络训练模块:结合自适应注意力卷积子模块和 GCN 子模块,学习图的全局和局部特征。

漏洞检测模块:融合学习到的特征,进行最终的漏洞分类。

VulATGCN 的实现过程如下:首先,输入的源代码经过规范化处理,去除噪声信息。然后,利用 Joern 工具从规范化代码中提取 CPG。接着,通过图向量嵌入将 CPG 转换为可学习的向量表示。在图特征提取阶段,计算图的中心性指标,进一步增强节点特征。随后,设计图神经网络训练模块结合自适应注意力卷积模块和 GCN 模块,学习图的全局和局部特征。最后,漏洞检测模块基于两个架构学习到的特征,输出最终的漏洞检测结果。

2.1 代码规范化方法

在软件源代码漏洞检测中,源代码丰富的语义信息可以帮助准确表达漏洞特征。本文专注于从源代码中直接检测漏洞。在这个过程中,源代码中的注释、复杂且不规范的变量和函数命名等噪声信息会影响模型理解代码特征和漏洞语义的能力。为了降低这些无关信息对代码语义的影响,本文引入代码规范化的预处理步骤。

```
static void DetectRunCleanup(*det_ctx, *p, *const pflow)
{
    PACKET_PROFILING_DETECT_START(p,
    PROF_DETECT_CLEANUP);
    /* Cleanup pkt-specific part of the patternmatcher */
    PacketPatternCleanup(det_ctx);
    if(pflow != NULL)
    {
        /* update inspected tracker for raw reassembly */
        if(p->proto == IPPROTO_TCP && pflow->protectx !=
        NULL)
        {
            StreamReassembleRawUpdateProgress(pflow->protectx,
            p, det_ctx->raw_stream_progress);
            DetectEngineCleanHCBDuffers(det_ctx);
        }
    }
    PACKET_PROFILING_DETECT_END(p, PROF_DETECT_CLEANUP);
}
```

源代码

步骤1: 去除注释信息

```
static void DetectRunCleanup(*VAR1, *VAR2, *const
VAR3)
{
    PACKET_PROFILING_DETECT_START(VAR2, VAR4);
    PacketPatternCleanup(VAR1);
    if(VAR3 != NULL)
    {
        if(VAR2->VAR5 == VAR6 && VAR3->VAR7 !=
        NULL)
        {
            StreamReassembleRawUpdateProgress(VAR3->
            VAR7, VAR2, VAR1->CAR8);
            DetectEngineCleanHCBDuffers(VAR1);
        }
    }
    PACKET_PROFILING_DETECT_END(VAR2, VAR4);
}
```

步骤2: 变量名统一化

步骤3: 函数名统一化

图 2 代码规范化过程

Fig. 2 Code standardization process

用解析器将源代码转换为AST,并依据从AST中提取的控制流信息,构建控制流图,包括代码的条件语句和循环等控制结构;然后分析代码中的数据依赖关系,生成数据流图,显示变量的定义和使用情况,以及数据在代码中的流动路径;最后将AST、CFG和数据依赖图(Data Dependence Graph, DDG)进行融合,得到CPG。以上代码例子的CPG简化图形化表示如图3所示,不同颜色的边表示不同的依赖关系。

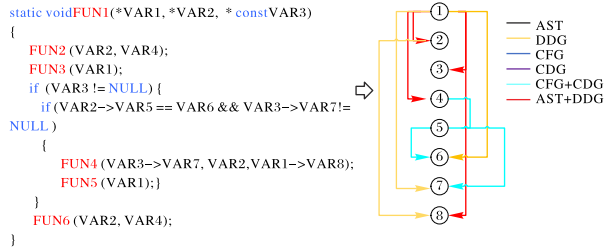


图3 CPG转化过程

Fig. 3 CPG transformation process

本文利用开源的静态代码解析工具Joern提取CPG。Joern工具无需预编译即可完成代码解析和CPG生成,支持多种编程语言,适用于任意粒度的待测代码,极大地降低了转换过程中的使用成本。

2.3 图向量嵌入技术

转换为DOT格式的图属性图中的节点表示函数的代码行,规范化的代码行作为节点的标签属性需要被嵌入表征才能作为后续模型训练过程的输入,因此将节点进行向量化表征是模型训练前的关键预处理步骤。本文使用预训练的CodeBERT模型对节点进行嵌入表征,以捕捉标签的语义信息。

具体来说,本文首先将每个节点中的标签作为一个句子提取,然后使用CodeBERT的tokenizer对句子进行分词处理,将句子分解为token列表,最后将token列表输入CodeBERT模型中,获取每个节点标签的向量表示。对于CPG的边特征,涉及将多重图转换为有向图的过程。本文使用NetworkX库读取以DOT格式存储的图数据。由于原始数据是多重图,首先需要将多重图转换为有向图。这个转换过程涉及合并多重边,并将多重边的属性合并为单个边的属性。边属性包括AST、CFG、DDG、控制依赖图(Control Dependence Graph, CDG)4种label属性。由于在CPG多重图中,两个点之间的边是不唯一的,即一条由点1指向点2的边可能是表示具有语法关系的label=AST的边,同时也是具有数据流关系的label=DDG的边。因此本文针对边特征进行编码,以AST、CFG、DDG、CDG的顺序针对每条边生成一个四维向量,取1则表示对应两个节点之间存在该种类型的边。

2.4 图特征提取技术

除了节点和边本身的label属性特征外,研究代码CPG结构本身具有的图特征同样是本文的重点研究内容。图的中心性指标基于节点在网络中的重要性和影响力来评估关键节点在函数中扮演的角色。为了捕捉可能与漏洞特征相关的图节点特征,在本文采用以下3种中心性指标:

1)特征向量中心性:在漏洞函数中,具有高特征向量中心性的节点通常是连接到许多其他重要节点的节点。这样的节点可能代表着漏洞函数中关键的执行路径或者控制流程。因此,特征向量中心性可以帮助识别出在漏洞函数中潜在的关键节点。

2)Katz中心性:Katz中心性考虑节点通过所有路径的重要性,路径长度越长的节点影响权重越小。因此Katz中心性可以帮助找到漏洞函数中连接在网络中距离较远但影响仍

然显著的关键节点。

3)中介中心性:在漏洞检测过程中,高中介中心性的节点可能是连接漏洞函数中重要模块或者关键功能路径上的节点,攻击者可能会利用这些节点来传播恶意代码或者影响漏洞函数的执行流程。因此,中介中心性可以帮助识别出与漏洞利用或者引入相关的重要节点。

图特征提取过程中将对CPG中每个节点进行中心性指标的计算并嵌入得到节点的中心性度量向量,同时对于每个节点,将编码后点标签特征与点中心性度量向量链接,得到最终的点特征矩阵。具体如算法1所示。

算法1 图特征提取。

输入 代码属性图 $G=(V,E)$;

输出 节点特征矩阵 N 。

计算图 G 中节点的特征向量中心性函数 $F_{\text{eigenvector_centrality}}$

计算图 G 中节点的Katz中心性函数 $F_{\text{katz_centrality}}$

计算图 G 中节点的中介中心性函数 $F_{\text{betweenness_centrality}}$

1)初始化空的节点特征列表 $V_{\text{node_features}}$

2) for each node $v \in V$ do

3) $v_{\text{label}} \leftarrow$ 获取节点 v 的标签

4) $v_{\text{encoded_label}} \leftarrow$ 使用CodeBERT模型编码label

5) $v_{\text{centrality_features}} \leftarrow [F_{\text{eigenvector_centrality}}[v], F_{\text{katz_centrality}}[v], F_{\text{betweenness_centrality}}[v]]$

6) $v_{\text{node_features}} \leftarrow$ 连接($v_{\text{encoded_label}}, v_{\text{centrality_features}}$)

7) end for

8) $N \leftarrow$ 将 $v_{\text{node_feature}}$ 转换为矩阵形式

9) return N

考虑到实际检测效果和性能,本文设置最终每个点的特征为700维。同时由于中心性分析将消耗大量计算资源,为了平衡计算资源,设置特征向量中心性计算函数的容忍度参数为0.001, Katz中心性计算函数的衰减因子为0.2。

2.5 图神经网络训练模块

传统的图神经网络模型虽然能处理图结构数据,但在处理长距离依赖和全局信息时存在局限性。另一方面,Transformer模型在处理序列数据和捕获长距离依赖关系方面表现出色,但直接应用于图结构数据时效果不佳。因此,本文结合Transformer和GCN进行图神经网络训练模块设计。

图神经网络训练模块的核心思想是结合传统GCN局部特征提取和Transformer捕获长距离依赖的优势,有效地学习和表示代码的结构化特征。

该模块主要由自适应注意力卷积模块和GCN模块两个子模块组成。

自适应注意力卷积模块是图特征神经网络学习模块的核心组成。该模块首先将输入的图特征(包括点特征、边特征)进行线性变换,定义如式(1)所示:

$$\begin{cases} q = W_q(x) \\ k = W_k(x) \\ v = W_v(x) \end{cases} \quad (1)$$

其中: q 表示查询向量; k 表示键向量; v 表示值向量; x 表示输入节点特征矩阵; W_q 表示查询权重矩阵; W_k 表示键权重矩阵; W_v 表示值权重矩阵。

然后,通过full_attention_conv函数计算注意力权重,计算过程涉及所有节点,使每个节点可以与图中的所有其他节点进行交互,而不仅仅是直接相连的节点。计算后注意力权重动态调整,模型对不同节点的关注程度进行重新分配。full_attention_conv函数如式(2)所示:

$$\text{full_attention_conv}(Q, K, V, d_k) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

其中: Q 表示查询矩阵; K 表示键矩阵; V 表示值矩阵; d_k 表示键向量的维度; softmax 表示对矩阵的每一行应用的 softmax 函数。

同时自适应注意力卷积模块采用多头注意力机制并堆叠多个自适应注意力卷积层, 允许信息在更深的层次上传播, 进一步增强模型捕获不同类型长距离依赖的能力。最后, 自适应注意力卷积层通过残差连接增强自适应能力, 允许模型在深层网络中保持对原始输入的关注, 同时学习更复杂的特征表示, 如式(3)所示:

$$\text{output} = \alpha \times \text{attention_output} + (1 - \alpha) \times \text{input} \quad (3)$$

其中: output 表示自适应注意力卷积层的最终输出; attention_output 表示通过自注意力机制计算得到的输出; input 表示自适应注意力卷积层的原始输入; α 表示一个可调节的参数, 用于平衡注意力输出和原始输入。

通过以上的自适应注意力分配机制, 在保留了整个特征图的长距离依赖的同时, 根据图特征(例如边权重特征)为图像不同区域重新分配关注度, 为在显著区域实现更细粒度的局部特征分析提供可能; 同时在不显著区域的粗粒度分析也将提高模型运行效率。例如, 在处理一个可能存在缓冲区溢出漏洞的代码片段时, 自适应注意力卷积层可能会更多地关注内存分配和访问相关的节点, 而对于其他不相关的代码部分给予较少的注意力。这种自适应的注意力分配机制使模型能更准确地捕获潜在的漏洞模式。

同时, GCN 模块作为自适应注意力卷积模块的补充, 专注于提取局部图结构特征。在实际应用中, GCN 可能更擅长识别某些特定类型的漏洞, 例如那些主要由局部代码结构决定的漏洞类型。通过聚合邻居节点的信息, GCN 能有效地捕获代码中的局部依赖关系, 这对于检测变量未初始化、空指针引用等局部性强的漏洞尤为重要。

GCN 模块的工作原理是通过聚合邻居节点的信息更新中心节点的表示, 如式(4)所示:

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(l)} \cdot \mathbf{W}^{(l)}) \quad (4)$$

其中: $\tilde{\mathbf{A}}$ 表示归一化的邻接矩阵; $\mathbf{H}^{(l)}$ 表示第 l 层的节点特征; $\mathbf{W}^{(l)}$ 表示可学习的权重矩阵; σ 表示非线性激活函数。

在本模块的 GCN 设计中, 第一层初始化层接收初始的节点特征和图的结构信息(边索引和边权重)后, 使用一个图卷积层将输入特征进行一次卷积运算, 并对卷积后的特征进行批归一化, 随后对归一化后的特征进行非线性激活, 引入非线性, 使模型能表示更复杂的特征关系。同时将对激活后的特征应用 Dropout, 防止过拟合。

随后在中间层中添加多个隐藏层, 进行特征深化, 逐层提取更深层次的特征。每个隐藏层接收来自上一层的节点特征后, 对节点特征进行逐层卷积运算。同时每一层卷积后都进行批归一化和激活操作保证特征在多层传递中的稳定性, 并应用 Dropout 进行特征随机失活。

最后在输出层接收来自最后一层隐藏层的节点特征, 使用最后一层全连接层将特征映射到输出通道, 得到经过 GCN 处理的图局部特征。

2.6 基于融合特征漏洞检测技术

漏洞检测模块的核心思想是将图特征神经网络提取的特征进行融合实现漏洞检测。该模块的创新点在于其灵活的特征融合策略和图级别的池化操作, 可以根据具体任务的需求选择使用加权和融合或拼接融合的方式来结合自适应注意力卷积模块和 GCN 模块的输出。

加权和融合函数如式(5):

$$\mathbf{x} = \omega \times \mathbf{x}_{\text{GCN}} + (1 - \omega) \times \mathbf{x}_{\text{自适应注意力卷积}} \quad (5)$$

拼接融合函数如式(6):

$$\mathbf{x} = \text{concat}(\mathbf{x}_{\text{GCN}}, \mathbf{x}_{\text{自适应注意力卷积}}) \quad (6)$$

其中: ω 表示融合权重; $\mathbf{x}$ 表示拼接后的特征向量; $\mathbf{x}_{\text{GCN}}$ 表示通过 GCN 模块提取的特征; $\mathbf{x}_{\text{自适应注意力卷积}}$ 表示通过自适应注意力卷积模块提取的特征; $\text{concat}()$ 表示拼接函数。

这种设计使模型能在不同类型的漏洞检测任务中自适应地调整特征融合策略。例如, 对于需要同时考虑全局和局部信息的复杂漏洞类型, 拼接融合可能会表现得更好; 而对于某些主要依赖于全局或局部信息的漏洞类型, 加权和融合可能更适合。

在特征融合之后, 由于用于训练测试的源代码的长度不同, 生成的 CPG 大小不同, 图特征向量的长度也并不统一, 因此采用全局平均池化将图中所有节点的信息整合到一个固定长度的全局特征向量中, 使模型对图的尺度变化具有不变性, 提高模型的泛化能力和鲁棒性。全局平均池化的函数如式(7)所示:

$$\text{GAP}(d) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \mathbf{x}(i, j, d) \quad (7)$$

其中: $\text{GAP}(d)$ 表示输入张量的第 d 个特征图的全局平均池化结果; H 表示特征图的高度; W 表示特征图的宽度; $\mathbf{x}(i, j, d)$ 表示输入张量中第 d 个特征图在位置 (i, j) 处的值。

通过全局平均池化, 模型综合考虑所有节点的信息, 形成一个全面的代码片段表示。这种表示方式特别适合那些需要考虑整体代码结构和语义的漏洞类型, 如逻辑错误或设计缺陷导致的安全漏洞。最后, 模块通过一个全连接层进行二分类, 将图级特征映射到二维输出空间, 最终输出结果表示是否存在漏洞的概率。

3 实验与结果分析

3.1 数据集

本文的实验数据来源于漏洞数据集 Big-Vul^[7]。Big-Vul 数据集是一个 C/C++ 代码真实漏洞数据集, 涵盖了 3 754 个代码漏洞, 涉及 91 种不同的漏洞类型, 共有 11 834 个漏洞函数。同时, 为了方便与领域内其他方法做性能对比, 且为了均衡漏洞数据和无漏洞数据的数量, 无漏洞函数选用另一个漏洞数据集 SARD, 选取 10 000 个无漏洞函数。SARD 包含大量的构造的人工漏洞函数和无漏洞函数, 是目前最大也是最常用的数据集之一。

需要注意的是, Big-Vul 数据集基于爬虫方法从公开的 CVE (Common Vulnerabilities and Exposures) 数据库和与 CVE 相关的源代码存储库中爬取数据, 标注补丁前的函数为漏洞函数, 标注补丁后的函数为无漏洞函数。而在本文实际使用数据集的过程中发现部分函数并未与具体的 CVE 编号相关联, 本文认为这部分数据由于无法标注缺乏真实性而需要过滤。因此, 本文最终使用了 Big-Vul 数据集中的 9 984 个漏洞函数。

3.2 实验环境和配置

实验在配备 128 GB 内存、16 核英特尔 Xeon 处理器和 16 GB 显存的英伟达 RTX A4000 系列显卡的 Linux 服务器上进行; 数据集按照 8:1:1 随机划分为训练集、验证集和测试集; 代码基于 PyTorch、Numpy 等库进行开发。

3.3 评价标准

为评估漏洞检测模型的有效性, 本文采用以下标准对方法进行评估。

准确率 (accuracy)、精确率 P 、召回率 R 和 F1 分数 (F1 score) 的公式如下:

$$V_{\text{accuracy}} = (TP + TN) / (TP + TN + FP + FN)$$

$$P = TP / (TP + FP)$$

$$R=TP/(TP+FN)$$
$$V_{F1}=2\times(P\times R)/(P+R)$$

其中: TP 为正确预测为有漏洞的样本数; TN 为正确预测为无漏洞的样本数; FP 为错误预测为有漏洞的样本数; FN 为错误预测为无漏洞的样本数量。

3.4 实验结果与分析

3.4.1 超参数调参实验

为了获得最佳的模型检测性能,首先对神经网络参数进行超参数调参实验,使用网格搜索方法进行超参数调优,调优过程中,使用交叉熵损失函数,优化器为 Adam,调优的参数、范围和梯度如表 1 所示。

表 1 超参数及其取值范围和梯度
Tab. 1 Hyperparameters and their ranges and ingredients

超参数	取值	梯度
GCN	hidden_channels	[64,512]
	num_layers	[2,5]
	dropout	[0.1,0.5]
自适应注意力卷积层	hidden_channels	[64,512]
	num_layers	[2,5]
	dropout	[0.1,0.5]
	num_heads	[1,8]
	alpha	[0.2,0.8]
	graph_weight	[0.2,0.8]
	aggregate	add, cat
Adam 学习率	$10^{-4}, 10^{-3}, 10^{-2}$	—

其中 graph_weight 是模型有关平衡自适应注意力卷积模块和 GCN 模块对特征向量最终贡献的参数。当 graph_weight 取 0 时,完全使用自适应注意力卷积模块的输出向量,当 graph_weight 取 1 时,完全使用 GCN 的输出向量。同时由于部分参数与模型的计算效率相关,在超参数调优的过程中,时间开销也是一项重要参考。

最终根据网格搜索方法的结果同时综合时间消耗,最终参数使用分别为: GCN hidden_channels=128, GCN num_layers=3, GCN dropout=0.5, 自适应注意力卷积层 hidden_channels=64, 自适应注意力卷积层 num_layers=2, 自适应注意力卷积层 dropout=0.5, 自适应注意力卷积层 num_heads=3, 自适应注意力卷积层 alpha=0.5, 自适应注意力卷积层 graph_weight=0.8, 自适应注意力卷积层 aggregate=add, Adam 学习率为 10^{-3} 。

3.4.2 漏洞检测实验

本文将 VulATGCN 与几种先进的漏洞检测工具进行比较,包括 3 个基于规则的商用漏洞检测工具(Checkmarx^[16]、FlawFinder^[17]和 RATS^[18])和 7 个基于深度学习的漏洞检测方法(LineVul^[19]、LineVD^[9]、DeepLineDP^[20]、VulSniper^[21]、VulMPFF^[3]、MGVD^[10]和文献[4]方法)。实验结果如表 2 所示。

基于规则的方法是基于网络安全专家定义的漏洞相关模式直接对源代码进行匹配的漏洞检测方案。目前常用的漏洞相关模式通常与源代码的各种数值指标相关,例如循环复杂度^[22]、McCabe^[23]、复杂度指标和代码变更度量^[24]等。利用软件度量的漏洞检测方法基于以下假设:复杂的代码难以理解,因此也难以维护和测试,从而更容易包含漏洞。同样,频繁修改的代码更易出错,因此更可能存在漏洞。尽管这些度量在测量软件质量以及缺陷预测方面已有长久应用,但它们并不是直接对应代码安全的概念。事实上,易被攻击的代码块并不总是复杂或庞大。因此,仅基于这些度量的方法在

漏洞检测方面通常表现一般。如表 2 中精确率、召回率和 F1 分数所示,Checkmarx、FlawFinder 和 RATS 的检测效果均不理想。从代码审计的视角看,诸如代码复杂性这样的因素可作为漏洞的部分考量,但通常不是判断代码存在漏洞的决定性因素。不同于基于规则的方法,VulATGCN 结合 Transformer 自注意力机制和 GCN 模型自动学习训练样本中的漏洞模式,达到了更低的漏报率和误报率。

表 2 不同方法的实验结果对比 单位: %
Tab. 2 Comparison of experimental results of different methods unit: %

方法	准确率	精确率	召回率	F1 分数
Checkmarx ^[16]	30.1	42.1	26.3	33.0
FlawFinder ^[17]	38.2	39.9	30.5	34.8
RATS ^[18]	39.3	40.8	32.9	35.7
LineVul ^[19]	56.2	61.7	68.2	64.8
LineVD ^[9]	59.6	63.4	75.8	69.3
DeepLineDP ^[20]	54.7	63.2	69.4	66.2
VulSniper ^[21]	59.0	60.4	71.2	65.5
VulMPFF ^[3]	94.2	30.7	42.4	35.6
MGVD ^[10]	83.1	24.1	74.2	36.4
文献[4]方法	72.9	70.5	80.3	75.1
VulATGCN	68.5	70.7	98.0	82.9

基于 token 序列表征源代码的方法将源代码按照特定分词规则方法拆解为 token 序列后进行模型训练。在代码编译过程中,源代码通过编译器处理,从连续的字符流中分隔出不同 token,这些 token 是代码的最小语法单位。编译器视角下的代码是一连串具有特定含义的 token。因此,部分研究基于一个假设:通过特定规则提取源代码中的词法关系和语义信息,将源代码转换为 token 序列,是对源代码的一种合理抽象表示。对比方法中 LineVul 即使用 token 序列表征源代码。LineVul 首先通过词法分析将源代码转换为 token 序列,然后使用预训练的 BERT 模型做向量嵌入和漏洞检测。因为 LineVul 将源代码视为类似自然语言的纯文本,且直接使用 BERT 进行预测,缺乏对代码语言丰富结构信息的考量而损失了代码中的大量关键信息,因此检测效果较差。

基于图表征源代码的方法是将源代码处理为代码不同图表征后进行模型训练的方案。与 token 序列表征相比较,图表征是更抽象复杂的表征形式:token 序列表征无法保留源代码的复杂结构,表征的语义依赖于最终得到的 token 序列中的单一顺序,因此通常只能表征线性语义;而图表征可以表示顶点(即语句或 token)之间的多对多联系,可以体现程序中元素之间的复杂结构关系。同时由于代码的图表征种类多样,不同图种类对图的表示能力也不同。

其中:DeelLineDP 使用 AST 表征源代码,LineVD 和文献[4]使用 PDG 表征源代码。由于 AST 更倾向体现源代码的语法结构,而当前漏洞通常与源代码语义信息联系更紧密,因此 DeepLineDP 检测效果不理想。PDG 虽然可以表示源代码中的各种依赖关系,但缺少对语法关系和节点(即语句或 token)语义的刻画,因此 LineVD 和文献[4]检测效果较于 DeepLineDP 有所进步但仍然具有提升空间。

MGVD 使用 AST、PDG 和 token 分别表示源代码,并将 3 种表征方式表征为一个三通道特征矩阵进行漏洞的识别。由于 MGVD 方法并未开源,因此 VulATGCN 直接和 MGVD 在 Big-Vul 数据集上的实验结果作对比,结果显示 VulATGCN 在精确率、召回率和 F1 分数上都略优于 MGVD。

VulSniper 和 VulMPFF 使用了 CPG 表征源代码并设计神

经网络结合注意力模块学习特征权重进行漏洞检测。但是 VulSniper 和 VulMPFF 简单的注意力模块无法捕获长距离依赖关系的能力,同时也无法根据图特征分配对图中不同部分的注意力权重,忽略了 CPG 中重要的局部特征和全局图特征,导致 VulSniper 和 VulMPFF 在 Big-Vul 完整数据集上检测效果不佳,精确率、召回率和 F1 分数都低于 VulATGCN。

综上,VulATGCN 在 Big-Vul 和 SARD 数据集上平均 F1 分数为 82.9%,相较于对比的 7 种基于深度学习的漏洞检测方法,F1 分数提高了 10.4%~132.9%,平均提高了约 52.9%,具有明显的优势。

3.4.3 消融实验

为了评估 VulATGCN 中各个组件的贡献,本文设计了一系列消融实验。消融实验设计实验组如下:

基线:使用 word2vec 进行节点嵌入+传统 GCN+无中心性分析;

对照组 1:使用 word2vec 进行节点嵌入+传统 GCN+有中心性分析;

对照组 2:CodeBERT 进行节点嵌入+传统 GCN+无中心性分析;

对照组 3:word2vec 进行节点嵌入+结合 AT-GCN 模型+无中心性分析;

对照组 4:CodeBERT 进行节点嵌入+传统 GCN+有中心性分析;

对照组 5:CodeBERT 进行节点嵌入+结合 AT-GCN 模型+无中心性分析;

对照组 6:word2vec 进行节点嵌入+结合 AT-GCN 模型+有中心性分析;

对照组 7:CodeBERT 进行节点嵌入+注意力模型+有中心性分析;

VulATGCN:使用 CodeBERT 进行节点嵌入+结合 AT-GCN 模型+有中心性分析;

消融实验结果如图 4 所示。

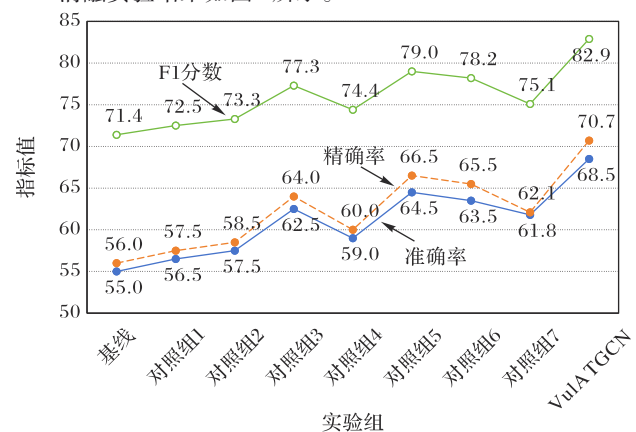


图4 消融实验结果

Fig. 4 Ablation experimental results

为了更直观地表现不同组件组合对最终实验性能的影响,消融实验结果可视化热力图展示,其中每个单元格代表特定配置下的性能指标与基线的差值。如图 5 所示。

根据热力图显示的消融结果,结合不同组件对具体性能指标的影响,逐一探讨节点嵌入方法的影响、中心性分析的作用、结合 AT-GCN 模型的优势和 3 个组件在整体实验效果中的贡献对比。

1) 节点嵌入方法的影响。

实验数据显示,使用 CodeBERT 进行节点嵌入(如对照组 2、对照组 4、对照组 5、VulATGCN)相较于使用 word2vec

(如基线、对照组 1、对照组 3、对照组 6),各项性能指标均有不同程度的提升。分析可知,word2vec 的设计是用于自然语言处理领域的词嵌入任务,而代码语言与自然语言相比,存在更多上下文语义关联,包括函数调用、变量作用域等。因在大规模代码语言上预训练的 CodeBERT 模型具备强大的上下文感知能力,在代码图节点的嵌入上明显优于仅仅基于单词共现的 word2vec。

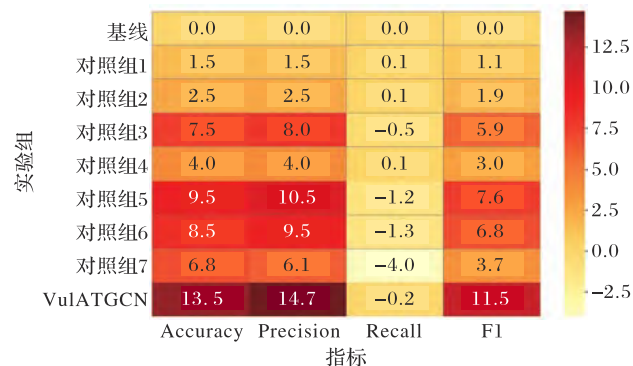


图5 消融实验的热力图

Fig. 5 Heatmap of ablation experiment

2) 中心性分析的作用。

从热力图中可以观察到,通过中心性分析为节点附加图的全局特征以实现与节点的局部特征互补,提供额外的图特征洞察,也对性能的提升起到了关键作用。从对照组 1 与基线的对比、对照组 4 与对照组 2 的对比、VulATGCN 与对照组 5 的对比中可见,引入中心性分析对准确率和精确率带来了 1.5 到 4.2 个百分点的提升。因此为每个节点提供它在整个图中的重要性信息,帮助识别 CPG 的关键节点,突出节点在图结构中的不同重要程度有益于代码整体特征的表达。

3) 结合 AT-GCN 模型的优势。

由热力图可见,结合 AT-GCN 模型的组件对性能整体的提升最大。结合 AT-GCN 模型(对照组 3、对照组 5、对照组 6、VulATGCN)相较于仅使用传统 GCN 的模型(基线、对照组 1、对照组 2、对照组 4),在精确率和准确率上分别最多提升了 10.7 和 9.5 个百分点;同时,结合 AT-GCN 模型的 VulATGCN 对比仅使用注意力模型的对照组 7 也有着 6.7 和 8.6 个百分点的提升。

对照组 4、对照组 7 和 VulATGCN 的对比显示了结合 AT-GCN 模型在整体性能上的优越性,尤其是在对复杂代码图进行灵活而多维度的特征分析时,结合 VulATGCN 模型的优势尤为突出。因此,基于图特征去反映图中不同节点和边的显著程度,并以此使用自适应的注意力分配机制,对图中的显著和不显著区域进行不同粒度的分析与特征提取,精确分析代码图辅助模型进行分类是有效的。同时表明,使用 GCN 的强局部结构分析和特征提取能力弥补 Transformer 善于捕捉全局关系而对局部结构分析能力欠缺的方法是可行的。

4) 3 个组件在整体实验效果中的贡献对比。

结合 AT-GCN 模型在消融实验中对整体性能的提升最明显,而节点嵌入方法和中心性分析尽管各自提供了不同程度的提升,但是效果较弱。

而中心性分析带来的性能提升与不同节点嵌入方法的选用带来的提升较为接近,远小于模型改变带来的提升。经分析,这是由于模型的总体实现中包括 GCN,而 GCN 已经具有一定的图结构信息捕捉能力,中心性分析提供的特征与作为基线方法的 GCN 模型捕捉到的特征存在部分重复,进而导致性能的提升有限。

结合 AT-GCN 模型的对照组,对于所有主要性能指标,如准确率、精确率和 F1 分数,都表现出了最高的提升幅度。这也说明,在基于深度学习的漏洞检测或源代码分析任务中,针对特定代码表征方法进行适配的模型设计与实现是所有步骤中最重要的因素。

4 结语

本文提出了一种基于代码属性图和 AT-GCN 的软件漏洞检测方法 VulATGCN。VulATGCN 通过 CPG 全面表示代码结构,利用 CodeBERT 捕捉语义信息,并结合图中心性分析提取高层次特征。AT-GCN 模型有效融合了 Transformer 的全局依赖捕捉能力和 GCN 的局部结构分析能力,实现了对代码特征的深入学习。实验结果表明,VulATGCN 在漏洞检测任务上取得了显著的性能提升。

尽管本文取得了令人鼓舞的结果,但仍存在一些局限性:

1)漏洞类型覆盖粗糙:未专门针对特定类型的 CWE 漏洞数据进行实验,不同类型的漏洞可能需要不同的注意力权重分配策略,即不同类型的漏洞可能需要不同参数设置的图神经网络模型,因此需要针对不同类型的 CWE 漏洞,开发对应的特征提取方法并调整模型参数;

2)数据集规模和多样性有限:尽管使用了真实漏洞数据集,但样本数量和种类较少,未来需要构建更大规模、更高质量、细粒度且具有准确标注的数据集,或考虑使用数据增强技术来扩充训练集,探索跨项目、跨语言的漏洞检测方法;

3)检测粒度:当前方法仍停留在函数级别的漏洞检测,无法精确定位到具体的漏洞行,开发设计能基于更细粒度的漏洞数据进行代码行级别漏洞检测定位的方法也是未来一个重点研究方向。

参考文献 (References)

- [1] PLATE H, PONTA S E, SABETTA A. Impact assessment for vulnerabilities in open-source software libraries [C]// Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution. Piscataway IEEE, 2015: 411-420.
- [2] CROFT R, BABAR M A, KHOLOSI M M. Data quality for software vulnerability datasets [C]// Proceedings of the IEEE/ACM 45th International Conference on Software Engineering. Piscataway: IEEE, 2023: 121-133.
- [3] CAO X, WANG J, WU P, et al. VulMPFF: a vulnerability detection method for fusing code features in multiple perspectives [J]. IET Information Security, 2024, 2024: No. 4313185.
- [4] 胡雨涛,王溯远,吴月明,等. 基于图神经网络的切片级漏洞检测及解释方法[J]. 软件学报, 2023, 34(6): 2543-2561. (HU Y T, WANG S Y, WU Y M, et al. Slice-level vulnerability detection and interpretation method based on graph neural network [J]. Journal of Software, 2023, 34(6): 2543-2561.)
- [5] YAMAGUCHI F, GOLDE N, ARP D, et al. Modeling and discovering vulnerabilities with code property graphs [C]// Proceedings of the 2014 IEEE Symposium on Security and Privacy. Piscataway: IEEE, 2014: 590-604.
- [6] FENG Z, GUO D, TANG D, et al. CodeBERT: a pre-trained model for programming and natural languages [C]// Findings of the Association for Computational Linguistics: EMNLP 2020. Stroudsburg: ACL, 2020: 1536-1547.
- [7] FAN J, LI Y, WANG S, et al. A C/C++ code vulnerability dataset with code changes and CVE summaries [C]// Proceedings of the IEEE/ACM 17th International Conference on Mining Software Repositories. New York: ACM, 2020: 508-512.
- [8] SAMATE. NIST Software assurance reference dataset [DS/OL]. [2024-06-12]. <https://samate.nist.gov/SRD/index.php>.
- [9] HIN D, KAN A, CHEN H, et al. LineVD: statement-level vulnerability detection using graph neural networks [C]// Proceedings of the 19th International Conference on Mining Software Repositories. New York: ACM, 2022: 596-607.
- [10] QIU F, LIU Z, HU X, et al. Vulnerability detection via multiple-graph-based code representation [J]. IEEE Transactions on Software Engineering, 2024, 50(8): 2178-2199.
- [11] LI Z, ZOU D, XU S. VulDeePecker: a deep learning-based system for vulnerability detection [C]// Proceedings of the 2018 Network and Distributed Systems Security Symposium. Reston, VA: Internet Society, 2018: 1-15.
- [12] ZOU D, WANG S, XU S, et al. μ VulDeePecker: a deep learning-based system for multiclass vulnerability detection [J]. IEEE Transactions on Dependable and Secure Computing, 2021, 18(5): 2224-2236.
- [13] DAM H K, TRAN T, PHAM T, et al. Automatic feature learning for predicting vulnerable software components [J]. IEEE Transactions on Software Engineering, 2021, 47(1): 67-85.
- [14] LI X, XIN Y, ZHU H, et al. Cross-domain vulnerability detection using graph embedding and domain adaptation [J]. Computers and Security, 2023, 125: No. 103017.
- [15] CHENG X, WANG H, HUA J, et al. DeepWukong: statically detecting software vulnerabilities using deep graph neural network [J]. ACM Transactions on Software Engineering and Methodology, 2021, 30(3): No. 38.
- [16] Checkmarx Ltd. Checkmarx [EB/OL]. [2024-03-19]. <https://www.checkmarx.com/>.
- [17] WHEELER D A. Flawfinder [EB/OL]. [2024-02-20]. <http://www.dwheeler.com/flawfinder/>.
- [18] Secure Software Inc. Rough Audit Tool For Security (RATS) [EB/OL]. [2024-03-19]. <https://code.google.com/archive/p/rough-auditing-tool-for-security/>.
- [19] FU M, TANTITHAMTHAVORN C. LineVul: a Transformer-based line-level vulnerability prediction [C]// Proceedings of the 19th International Conference on Mining Software Repositories. New York: ACM, 2022: 608-620.
- [20] PORNPRASIT C, TANTITHAMTHAVORN C K. DeepLineDP: towards a deep learning approach for line-level defect prediction [J]. IEEE Transactions on Software Engineering, 2023, 49(1): 84-98.
- [21] DUAN X, WU J Z, JI S, et al. VulSniper: focus your attention to shoot fine-grained vulnerabilities [C]// Proceedings of the 28th International Joint Conference on Artificial Intelligence. California: ijcai.org, 2019: 4665-4671.
- [22] DOYLE M, WALDEN J. An empirical study of the evolution of PHP Web application security [C]// Proceedings of the 3rd International Workshop on Security Measurements and Metrics. Piscataway: IEEE, 2011: 11-20.
- [23] McCABE T J. A complexity measure [J]. IEEE Transactions on Software Engineering, 1976, SE-2(4): 308-320.
- [24] NAGAPPAN N, BALL T. Use of relative code churn measures to predict system defect density [C]// Proceedings of the 27th International Conference on Software Engineering. New York: ACM, 2005: 284-292.

This work is partially supported by Henan Province Key Research and Development Program (221111210300).

LIANG Chen, born in 2000, M. S. candidate. His research interests include software component analysis.

WANG Yisen, born in 1990, Ph. D., associate professor. His research interests include cyber security.

WEI Qiang, born in 1979, Ph. D., professor. His research interests include software security, industrial control system security.

DU Jiang, born in 1990, Ph. D. candidate. His research interests include binary code similarity.